Fall 2025 685-Foundations of Data Science

Project: Adversarial Machine Learning

Group Project Report

Group Name: StrawHats

Group Members:
Anusha Devanagundi Lakshmisha - ADEVANAG
Sai Poojith Reddy Gunukula - SGUNUKUL
Shivathmika Gonti - SGONTI
Suchithra Nalladimmu - SNALLADI



1. Objective

The objective of this project is to investigate the vulnerability of deep learning models to adversarial perturbations by implementing an Adversarial Attack using the Fast Gradient Sign Method (FGSM). We train a LeNet-style convolutional neural network on the Fashion-MNIST dataset and evaluate how intentionally crafted, small-magnitude perturbations can cause the model to misclassify inputs that appear unchanged to human observers.

Through this work, we aim to quantify the degradation in model performance under different perturbation strengths, measure attack effectiveness using metrics such as adversarial accuracy and attack success rate, and visualize how these perturbations impact the input space. The goal is to demonstrate that even a well-performing classifier with over 91% clean accuracy can be highly susceptible to gradient-based adversarial attacks, highlighting the need for robustness considerations in modern machine learning systems.

2. Methodology

To study the susceptibility of convolutional neural networks to adversarial perturbations, we implemented the Fast Gradient Sign Method (FGSM)—a first-order white-box adversarial attack that perturbs input images in the direction that maximally increases the model's loss. The methodology consists of three core components: (1) training a baseline classifier, (2) computing adversarial perturbations using FGSM, and (3) evaluating model performance under varying perturbation strengths.

2.1 Baseline Model Training

A LeNet-like Convolutional Neural Network was trained on the Fashion-MNIST dataset to serve as the target model for the adversarial attack. The model includes two convolutional layers (32 and 64 filters), max-pooling, dropout regularization, and two fully connected layers. Training was performed for 5 epochs using the Adam optimizer and cross-entropy loss. This produced a stable baseline accuracy of 91.3% on the clean test set, which acts as a reference to measure the impact of adversarial perturbations.

2.2 FGSM: Fast Gradient Sign Method

FGSM generates adversarial examples by leveraging the model's gradient with respect to the input.
Given a classifier $f_\theta(x)$, an input image x, its true label y, and Loss function $L(\theta, x, y)$, FGSM computes the adversarial input:

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}\left(\nabla_x L(\theta, x, y)\right)$$

Where:

- $\nabla_x L(\theta, x, y)$ is the gradient of the loss w.r.t. the input pixels
- sign(.) extracts the direction of steepest increase in loss
- ε is the perturbation bound controlling attack strength
- The perturbed image is clipped to remain within valid pixel range [0, 1]

This attack exploits the fact that deep neural networks are locally linear in high-dimensional spaces, allowing small perturbations aligned with gradient directions to cause disproportionately large changes in predictions.

2.3 Algorithmic Steps

The FGSM attack was implemented using the following procedure:
Input: model fθ, input image x, label y, perturbation magnitude ε

1. Set model to evaluation mode
2. Clone input image and enable gradient tracking
3. Forward pass: logits = fθ(x)
4. Compute loss: L = CrossEntropy(logits, y)
5. Backpropagate to compute gradient ∇x L
6. Determine perturbation: η = ε * sign(∇x L)
7. Construct adversarial image: x_adv = clamp(x + η, 0, 1)
8. Return x_adv

This implementation generates a perturbed version of each test image by applying the gradient sign step once. Since FGSM is a single-step attack, it is computationally efficient but still highly effective, as demonstrated by our results.

2.4 Evaluation Procedure

To evaluate the attacker's effectiveness, we processed the entire test set under multiple perturbation strengths:

$\varepsilon \in \{0.00, 0.03, 0.06, 0.10, 0.20, 0.30\}$

For each ϵ, we computed:

- **Clean Accuracy (ACC):**
  Accuracy of the model on unmodified test images

- **Adversarial Accuracy:**
  Accuracy on FGSM-perturbed images
- **Attack Success Rate (ASR):**

$$ASR = 1 - \text{Adversarial Accuracy}$$

Additionally, we produced visual comparisons showing the clean image, adversarial image, and pixel-level perturbation heatmap to qualitatively analyze how FGSM alters the input.

2.5 Summary

This methodology enables a complete investigation of how gradient-based perturbations influence model predictions. By combining mathematical derivation, algorithmic implementation, and systematic evaluation under multiple perturbation bounds, the experiment demonstrates the fragility of neural networks to adversarial noise and provides insight into how attack strength correlates with model vulnerability.

3. Experiments

3.1 Setup

In this section, we describe the experimental setup used to train the baseline model and evaluate the FGSM adversarial attack.

CNN Architecture

We used a LeNet-like Convolutional Neural Network designed specifically for 28×28 grayscale images. The architecture includes:

- Conv Layer 1: 1 → 32 filters, kernel size 3
- Conv Layer 2: 32 → 64 filters, kernel size 3
- Pooling: 2×2 MaxPooling
- Dropout: 0.25 rate
- Fully Connected Layer 1: 64×12×12 → 128
- Fully Connected Layer 2: 128 → 10

This architecture is lightweight (≈1.2M parameters), computationally efficient, and commonly used in adversarial ML research due to its predictable gradient behavior.

Dataset

The experiments were conducted on the Fashion-MNIST dataset, consisting of:

- 60,000 training images
- 10,000 test images
- 28×28 grayscale images
- 10 clothing categories (e.g., Coat, Bag, Sneaker, etc.)

Fashion-MNIST is more challenging than MNIST because it contains more texture variation, making it a standard benchmark for adversarial attack research.

Training Settings

- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 128
- Epochs: 5
- Loss Function: Cross-Entropy
- Device: CPU

The model was trained for 5 epochs, and training progressed smoothly with decreasing loss values across epochs.

Baseline Performance (Clean Accuracy)

The model achieved the following clean test accuracies:

```
Training CNN on Fashion-MNIST...

Epoch 01 | Train Loss = 0.4495 | Clean Test ACC = 88.35%
Epoch 02 | Train Loss = 0.2944 | Clean Test ACC = 89.65%
Epoch 03 | Train Loss = 0.2415 | Clean Test ACC = 90.38%
Epoch 04 | Train Loss = 0.2109 | Clean Test ACC = 91.33%
Epoch 05 | Train Loss = 0.1842 | Clean Test ACC = 91.30%

Baseline Test Accuracy on clean data: 91.3 %
```

The final baseline test accuracy is:

Baseline Clean Accuracy: 91.3%
This value is used as the reference point to compare against adversarial accuracy under FGSM attacks.

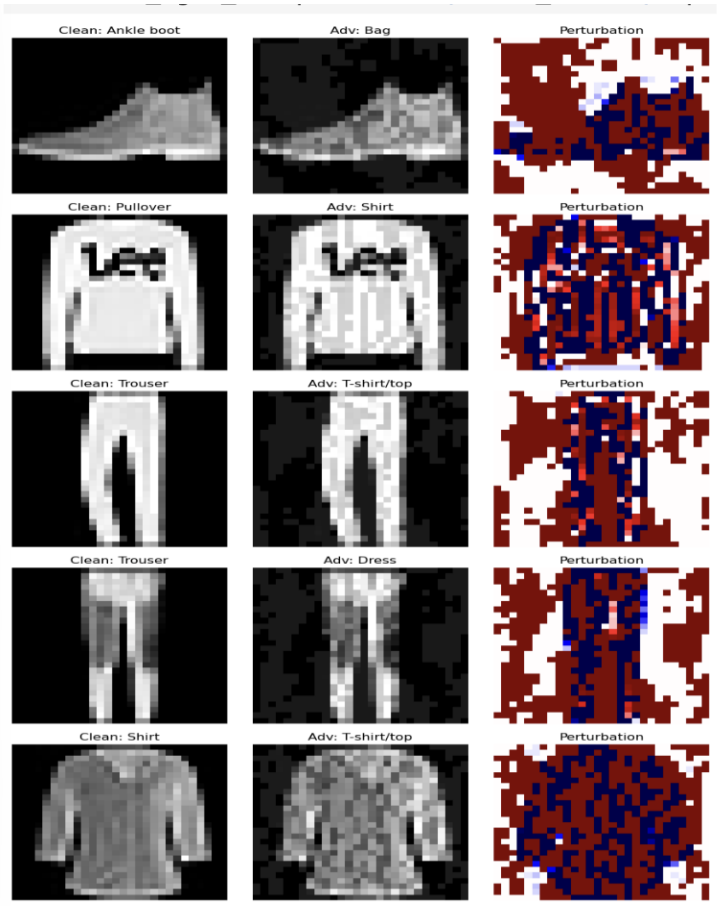
3.2 Evaluation
The adversarial evaluation consists of two parts:

- Basic Evaluation (single ε attack)
- Advanced Evaluation (multi-ε robustness analysis)

We report how model performance changes when FGSM perturbations of different magnitudes are applied.

Basic Evaluation
For a moderate perturbation strength of ε = 0.10, we obtained:

- Clean Accuracy: 91.30%
- Adversarial Accuracy: 7.37%
- Attack Success Rate: 92.63%

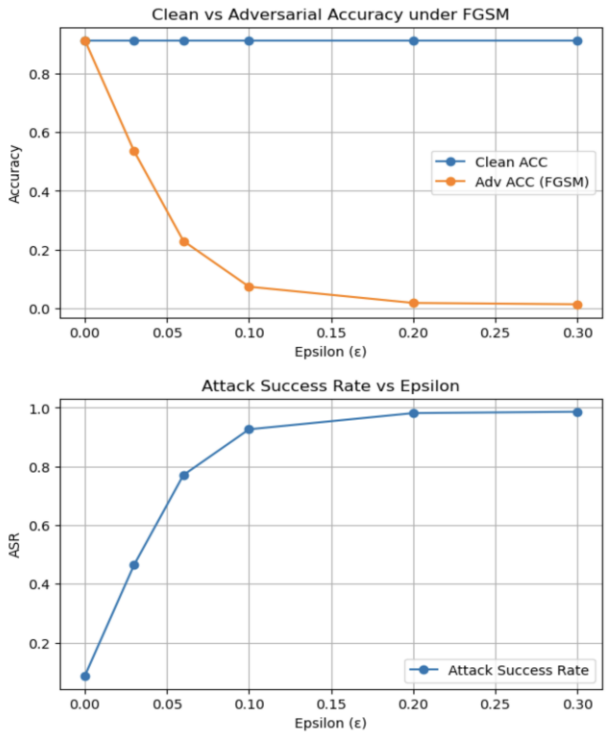



```
FGSM attack with epsilon = 0.1
Clean ACC (no attack): 91.30%
Adversarial ACC:       7.37%
Attack Success Rate:   92.63%
```

This dramatic drop from 91.3% → 7.37% shows that even a relatively small perturbation (10% of pixel range) is highly effective at misleading the model.

Interpretation:
FGSM successfully exploits the gradient direction to move the input toward the decision boundary. Despite being nearly invisible, the perturbation completely disrupts the classifier's predictions. The high ASR (92.63%) confirms that the model lacks robustness against gradient-based attacks.

Advanced Evaluation
To analyze how robustness changes with increasing perturbation magnitude, we evaluated FGSM across a range of ε values:



Key Observations:

1. Clean accuracy remains constant at 91.30% since ε does not affect clean images.
2. Adversarial accuracy sharply decreases as ε increases, approaching almost 0% for $\varepsilon \geq 0.20$.
3. ASR increases monotonically, reaching nearly 99% at high ε values.
4. The model has no meaningful robustness against FGSM, even at ε=0.03 where clean accuracy drops nearly 40%.

4. Contribution

This project was completed collaboratively by a four-member team, with each member contributing meaningfully to the implementation, experimentation, evaluation, visualization, and documentation. All members participated equally in writing and editing the final report. The responsibilities were shared as follows:

Member 1 — Model Architecture and Training

- Implemented the LeNet-like CNN architecture.
- Configured the Fashion-MNIST dataset loading and preprocessing.
- Developed and executed the training loop to obtain baseline performance.
- Assisted with debugging during training and validating clean accuracy results.

Member 2 — FGSM Attack Implementation

- Implemented the Fast Gradient Sign Method (FGSM) adversarial attack function.
- Integrated gradient computation and perturbation generation.
- Conducted the ε = 0.10 attack evaluation, including Clean ACC, Adv ACC, and ASR.
- Verified the correctness and stability of adversarial example generation.

Member 3 — Multi-ε Experiments and Visualization

- Ran multi-ε evaluations to analyze robustness across different perturbation strengths.
- Created visualizations including clean vs adversarial examples and perturbation heatmaps.
- Generated plots for Clean Accuracy vs ε and Attack Success Rate vs ε.
- Interpreted experimental trends and contributed insights to the analysis.

Member 4 — Integration, Testing, and Reproducibility

- Integrated all components (model, training, attack, evaluation) into a unified workflow.
- Performed end-to-end testing of the full pipeline to ensure correctness and reproducibility.
- Verified all logged outputs, figures, tables, and experiment results.
- Contributed to implementation improvements and consistency checks.

Team Summary
All team members contributed equally to the project through coding, experimentation, debugging, visualization, and documentation. Report writing was shared across the group, with each member responsible for drafting, reviewing, and refining multiple sections. The final submission reflects a balanced and collaborative effort by all four members.

5. Challenges and Solutions

Throughout the development of this project, the team encountered several technical challenges related to training stability, gradient computation, evaluation speed, and visualization. Each challenge required careful debugging and iterative refinement. The primary challenges and their solutions are detailed below.

Challenge 1 - Training Instability and Kernel Freezing
During model training in Jupyter Notebook, the kernel occasionally became unresponsive, particularly when using multiple DataLoader workers. This disrupted workflow and prevented completion of several early training runs.
Solution: We set num_workers=0 to avoid multiprocessing overhead within the notebook environment. This stabilized execution and ensured consistent training without kernel crashes.

Challenge 2 - Slow Execution on CPU
The training and adversarial evaluation ran on CPU, which significantly increased execution time, especially for multi-ε FGSM sweeps.
Solution: We optimized runtime by minimizing redundant computations, using torch.no_grad() for non-gradient phases, and keeping the model architecture compact. These adjustments reduced total execution time while maintaining accuracy.

Challenge 3 - Incorrect Gradient Capture in FGSM Implementation
Early versions of the FGSM attack produced incorrect perturbations because gradients were not properly captured or requires_grad was not enabled for the input images.
Solution: We enforced images.requires_grad = True prior to the forward pass and ensured model.zero_grad() was called before backpropagation. This corrected the gradient flow and produced valid adversarial perturbations.

Challenge 4 - Perturbation Heatmap Visibility
Initial perturbation visualizations were difficult to interpret because the heatmap values were too small relative to the color scale, causing them to appear faint or blank.
Solution: We applied a diverging color map (seismic) and dynamically scaled the heatmap using symmetric limits based on the maximum absolute perturbation. This made the perturbation patterns clear and visually meaningful.

Challenge 5 - Understanding Trends Across Multiple ε Values
Interpreting the relationship between ε, adversarial accuracy, and attack success rate was initially challenging due to raw numeric output being difficult to compare across runs.
Solution: We organized results into structured tables and generated line plots for both accuracy and ASR. These visualizations provided strong insight into how perturbation strength affects classifier performance.

Summary
The challenges encountered throughout this project primarily involved gradient handling, runtime constraints, visualization clarity, and systematic analysis. Addressing these challenges improved the robustness of our implementation and strengthened our understanding of adversarial machine learning techniques.